

LENGTH: say which spaces count

Check the value behind the display

01 / THE SAS EXAMPLE

```
data test;
  length text $10;
  input text $;
  len = length(text);
  datalines;
ABC
ABC
ABCDEFGHIJ
X
.
;
run;

proc print data=test noobs;
run;
```

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

Specify an invisible data detail and ask for a check that makes it visible.

Each text value occupies ten character positions. LENGTH ignores trailing blanks and returns 1 for all blanks. The period on the final input row supplies a missing character value.

EXPECTED RESULT

Row	Stored width	len
1	10	3
2	10	3
3	10	10
4	10	1
5	10	1

Both ABC rows are padded with seven blanks. The final row is ten blanks. SAS LENGTH returns 1 for this all-blank character value.

Expected values checked in Python; SAS execution pending.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.

Show code in this response. Do not create or run files, or read existing Python files.

LOOK AT THE RESPONSE

Which library and method did it choose? Did it preserve the expected behavior? Compare the result with page 1 before accepting it.

TRY IT / THREE FRESH ATTEMPTS

Run A three times with the same SAS example and setup. Save each first response before asking follow-up questions. Record agreement as well as differences.

Bedrock A/B trials for this example are pending.

WHY CHOICES REMAIN OPEN

1 An ordinary-looking request

A length calculation sounds simple, but the SAS source also specifies fixed-width storage.

2 Invisible choices

Python strings need not carry SAS padding. The model must choose how to represent and count those blanks.

3 A display can hide differences

Two printed ABC values can look identical. Inspect the stored value as well as the calculated length.

ANTI-PATTERN TO AVOID

Anti-pattern: asking to 'remove extra spaces' without saying which ones. Leading spaces can be meaningful. Specify trailing blanks for the calculation and preserve the stored text.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

```
Convert this SAS code to Python using pandas.
- Create text from ABC, ABC, ABCDEFGHIJ, X, and an empty string.
  The final SAS input period represents character missing here.
- Right-pad every text value with spaces to width 10.
- Preserve the stored text. For len, remove trailing ordinary
  spaces only with .str.rstrip(' '), then count characters.
- Match SAS LENGTH for a blank string: set its result to 1.
  Use .str.len().clip(lower=1) after the trailing-space removal.
- Print a visible representation of text and the len column.
- Include assertions: stored widths are all 10; len values are
  [3, 3, 10, 1, 1] in the original row order.
- Briefly explain why stored width and LENGTH can differ.

Show code in this response. Do not create or run files, or read
existing
Python files.
```

SAS REFERENCE

WHY THESE DETAILS HELP

- 1 Name the hidden detail**
Padding to ten characters makes the storage rule explicit and measurable.
- 2 Specify which spaces**
Remove only trailing ordinary spaces for the calculation. Keep stored text and leading spaces intact.
- 3 Include the blank case**
The last row checks LENGTH returns 1 for blanks. Trim-and-count alone would return 0.

CHECK IT YOURSELF

Run the code. Confirm every stored value has width 10 and len equals [3, 3, 10, 1, 1]. Confirm the calculation did not trim the stored text column.

When a result depends on invisible spaces, make the storage rule visible and verify it explicitly.